



Instituto Tecnológico de Costa Rica

Tarea #1 MiniPc

Estudiante:

Quiriat Mata Araya – 2023379891

Profesor:

Cristian Campos

Curso:

Principios de Sistemas Operativos

G60

Semestre II 2026

1. Objetivo del proyecto

Construir una aplicación de escritorio en Java que simule el funcionamiento de una computadora sencilla ("Mini PC"): un procesador con registros de propósito general, un acumulador, memoria principal dividida en espacio de Sistema Operativo y espacio de Usuario, y un Bloque de Control de Proceso (BCP) que describe al proceso en ejecución. El programa carga un archivo de texto plano (.asm) con instrucciones tipo ensamblador, valida su formato, lo traduce a su representación binaria y permite ejecutarlo paso a paso o de forma automática, mostrando en todo momento el estado del CPU, la memoria y el proceso.

2. Objetivos alcanzados

Frente a los requisitos del enunciado, el estado del proyecto es el siguiente:

- Aplicación 100% gráfica construida en Java.
- Carga de un archivo .asm seleccionado por el usuario mediante un selector de archivos.
- Validación del formato del archivo: se revisan todas las líneas y se reportan, de una sola vez, todos los errores encontrados (instrucción inválida, registro inválido, falta de valor inmediato, extensión de archivo incorrecta, entre otros).
- Ejecución paso a paso mediante el botón "Siguiente".
- Ejecución automática mediante el botón "Ejecutar todo" (corre el programa completo de una sola vez).
- Tamaño de memoria configurable por el usuario (mínimo 128 posiciones), con separación explícita entre espacio de Sistema Operativo y espacio de Usuario.
- Implementación del BCP (PCB) con sus atributos visibles en pantalla: PID, nombre del programa, estado, PC, dirección base, dirección límite, cantidad de instrucciones y una copia de los registros.
- Representación binaria de cada instrucción (8 bits de opcode/registro + 8 bits de operando en formato signo-magnitud), visible tanto en el panel de código como en el de memoria.

3. Arquitectura del proyecto

El código se organizó en tres paquetes, separando claramente los datos, la lógica de ejecución y la interfaz gráfica:

Paquete	Responsabilidad
com.tec.minipc.model	Estructuras de datos puras: Opcode, RegisterName, Instruction, Memory, Registers. No conocen la lógica de ejecución.

com.tec.minipc.core	Lógica del sistema: Assembler (lee y valida el .asm), AssemblyException, PCB (BCP) y Cpu (ciclo de ejecución).
com.tec.minipc.gui	Interfaz Swing:CodigoPanel, MemoriaPanel, RegistrosPanel, PcbPanel y MainFrame, que conecta todo con los botones de control.

4. Cómo ejecutar el programa

4.1 Desde NetBeans

1. Abrir el proyecto MiniPC en NetBeans.
2. Ubicar la clase Main.java dentro del paquete com.tec.minipc.
3. Clic derecho sobre Main.java → "Run File" (o seleccionar el archivo y presionar Shift+F6).
4. Se abre la ventana principal del simulador.

4.2 Uso de la interfaz

1. Presionar "Cargar .asm..." y seleccionar el archivo con el programa a ejecutar.
2. Opcionalmente, ajustar los campos "Memoria total" y "Espacio S.O." antes de cargar, para definir el tamaño de memoria y la separación entre zona de Sistema Operativo y zona de Usuario.
3. Si el archivo tiene errores de formato, se muestra un cuadro de diálogo con la lista completa de líneas inválidas y no se carga el programa.
4. Si el archivo es válido, se muestran: el programa cargado (con su binario) a la izquierda, el contenido completo de la memoria al centro, y los registros junto con el BCP a la derecha.
5. Presionar "Siguiente" para ejecutar una instrucción a la vez, observando cómo cambian los registros, el BCP y la posición resaltada en memoria y en el código.
6. Presionar "Ejecutar todo" para correr el resto del programa de forma automática.
7. Presionar "Reiniciar" para volver a cargar el mismo archivo desde cero y repetir la ejecución.

5. Formato de instrucciones y memoria

5.1 Set de instrucciones

El Mini PC reconoce cinco operaciones, cada una asociada a un código de 4 bits:

- 0001 — LOAD reg → AC = reg
- 0010 — STORE reg → reg = AC

- 0011 — MOV reg, N → reg = N
- 0100 — SUB reg → AC = AC - reg
- 0101 — ADD reg → AC = AC + reg

Los registros de propósito general se codifican también en 4 bits: AX = 0001, BX = 0010, CX = 0011, DX = 0100.

5.2 Codificación de una instrucción

Cada instrucción se representa con 2 bytes:

- Byte 1: 4 bits de operador + 4 bits de registro (ej. MOV AX → 0011 0001).
- Byte 2: valor entero en formato signo-magnitud de 8 bits — el bit más significativo indica el signo (0 = positivo, 1 = negativo) y los 7 bits restantes representan la magnitud. Solo se usa cuando la instrucción es MOV; en el resto vale 0.

Por ejemplo, "MOV BX, -8" se codifica como 0011 0010 10001000: opcode MOV (0011), registro BX (0010), signo negativo (1) y magnitud 8 (0001000).

5.3 Memoria: una instrucción por posición

A diferencia de un diseño donde cada byte ocupa una celda distinta, en este proyecto cada línea del programa ocupa una única posición de memoria (tal como lo pide el enunciado). Esa posición guarda la instrucción ya interpretada; su forma binaria (los 2 bytes descritos arriba) se calcula únicamente para mostrarla en pantalla, no para direccionar la memoria.

La memoria se divide en dos zonas contiguas: desde la posición 0 hasta osSize-1 para el Sistema Operativo (reservada, sin uso de programa de usuario), y desde osSize hasta el final para el espacio de Usuario, donde se carga el programa cuando se presiona "Cargar .asm...".

6. Componentes principales

6.1 Paquete model

- Opcode / RegisterName: enumeraciones con los códigos binarios de operaciones y registros, y su conversión desde/hacia texto.
- Instruction: parsea una línea de texto ("MOV AX, 5") a un objeto, y lo codifica/decodifica a sus 2 bytes binarios, incluyendo el signo-magnitud.
- Memory: arreglo de posiciones (una por instrucción), con las validaciones de tamaño mínimo y separación S.O./Usuario.
- Registers: banco de registros AC, AX, BX, CX, DX, PC e IR.

6.2 Paquete core

- **Assembler:** lee el archivo .asm línea por línea, ignora comentarios y líneas en blanco, y acumula todos los errores de formato encontrados antes de lanzar una única excepción con la lista completa.
- **AssemblyException:** excepción que agrupa todos los mensajes de error de validación.
- **PCB:** el Bloque de Control de Proceso — guarda PID, nombre del programa, estado (NUEVO, LISTO, EJECUTANDO, TERMINADO), PC, dirección base, dirección límite, cantidad de instrucciones y una copia de los registros.
- **Cpu:** implementa el ciclo Fetch-Decode-Execute, con avance paso a paso (step) o automático (runAll), y mantiene el PCB sincronizado después de cada instrucción.

6.3 Paquete gui

- **CodigoPanel:** tabla con el programa cargado (instrucción y binario), resaltando la instrucción actual.
- **MemoriaPanel:** tabla con todas las posiciones de memoria, distinguiendo con color la zona de Sistema Operativo y resaltando la posición donde apunta el PC.
- **RegistrosPanel / PcbPanel:** paneles de solo lectura que muestran en vivo los registros y los atributos del BCP.
- **MainFrame:** ventana principal; conecta los botones ("Cargar", "Siguiente", "Ejecutar todo", "Reiniciar") con el Assembler y el Cpu, y refresca los cuatro paneles después de cada acción.

7. Algoritmo clave: ciclo Fetch-Decode-Execute

El corazón del simulador es el método `step()` de la clase `Cpu`, que ejecuta un único ciclo de instrucción cada vez que el usuario presiona "Siguiente". A continuación, se muestra el código y su explicación.

```
public Instruction step() {
    if (terminado) return null;

    int direccionActual = registros.getPc();
    int limite = pcb.getDireccionBase() + pcb.getTamanoInstrucciones();

    if (direccionActual >= limite || memoria.isEmpty(direccionActual)) {
        terminado = true;
        pcb.setEstado(PCB.Estado.TERMINADO);
        return null;
    }

    // FETCH: se busca la instrucción en la posición actual del PC
    Instruction instruccion = memoria.read(direccionActual);

    // DECODE: se guarda en el registro IR y se avanza el PC
```

```
registros.setIr(instruccion);
registros.advancePc(1);

// EXECUTE: se aplica el efecto de la instrucción
ejecutar(instruccion);

pcb.actualizarDesde(registros);
return instruccion;
}
```

El método `ejecutar()`, llamado dentro del paso anterior, aplica la semántica de cada operación sobre el acumulador y los registros:

```
private void ejecutar(Instruction instruccion) {
    RegisterName reg = instruccion.getRegister();
    switch (instruccion.getOpcode()) {
        case MOV:
            registros.set(reg, instruccion.getOperand());
            break;
        case LOAD:
            registros.setAc(registros.get(reg));
            break;
        case STORE:
            registros.set(reg, registros.getAc());
            break;
        case ADD:
            registros.setAc(registros.getAc() + registros.get(reg));
            break;
        case SUB:
            registros.setAc(registros.getAc() - registros.get(reg));
            break;
    }
}
```

El límite de ejecución no se calcula únicamente revisando si una celda está vacía, sino a partir de los datos del propio BCP (dirección base más cantidad de instrucciones). Esto permite que el CPU sepa con exactitud dónde termina el proceso actual, independientemente del contenido que pueda quedar en memoria.

8. Flujo de ejecución completo

Desde que el usuario elige un archivo hasta que ve los resultados en pantalla, el flujo es el siguiente:

1. El usuario presiona "Cargar .asm..." y selecciona un archivo.
2. `Assembler.loadFromFile()` lee el archivo línea por línea, descarta comentarios y líneas vacías, y valida cada línea de código con `Instruction.parse()`. Si alguna línea no cumple el formato, se acumula el error, pero la lectura continúa para reportar todos los errores juntos.

3. Si hay errores, se lanza `AssemblyException` con la lista completa y `MainFrame` la muestra en un cuadro de diálogo, sin modificar el estado anterior de la aplicación.
4. Si el archivo es válido, se crea una nueva `Memory` con el tamaño configurado, y `Memory.loadProgram()` coloca cada instrucción en una posición consecutiva del espacio de Usuario.
5. Se crean nuevos `Registers` (con `PC` apuntando a la dirección base del programa) y un nuevo `PCB` en estado LISTO.
6. Se crea el `Cpu`, que queda listo para ejecutar.
7. Los cuatro paneles (código, memoria, registros, BCP) se refrescan para reflejar el estado inicial.
8. Cada clic en "Siguiente" llama a `Cpu.step()`: se hace fetch de la instrucción en la posición del `PC`, se decodifica, se ejecuta y se actualiza el BCP; luego `MainFrame` refresca los cuatro paneles y resalta la posición actual tanto en el código como en la memoria.
9. "Ejecutar todo" llama a `Cpu.runAll()`, que repite el paso anterior hasta que no queden instrucciones, y refresca los paneles una sola vez al finalizar.
10. Cuando el `PC` alcanza el límite del programa, el `PCB` pasa a estado TERMINADO y los botones de ejecución se deshabilitan hasta que se cargue un nuevo archivo o se reinicie.

9. Manejo de errores y validación

El programa valida el archivo `.asm` en varios niveles antes de permitir su ejecución:

- Extensión de archivo: se rechaza cualquier archivo que no termine en `.asm`.
- Formato de línea: cada línea debe ser un mnemónico reconocido (`LOAD`, `STORE`, `MOV`, `ADD`, `SUB`) seguido de un registro válido (`AX`, `BX`, `CX`, `DX`), y en el caso de `MOV`, seguido de una coma y un valor entero.
- Rango del valor inmediato: los valores de `MOV` deben poder representarse en el formato signo-magnitud de 8 bits (-127 a 127).
- Capacidad de memoria: si el programa tiene más instrucciones de las que caben en el espacio de Usuario configurado, se rechaza la carga con un mensaje explicativo.

En todos los casos, los errores se muestran en un cuadro de diálogo y el estado previo de la aplicación se conserva intacto: una carga fallida nunca deja la interfaz a medio actualizar.

10. Repositorio

Link: https://github.com/Q-ARAYA/Tarea1_PrincipiosOS.git